

Sistema Completo de Evolução de Sessão

Visão Geral

Criar uma interface robusta para evolução de sessões que permita ao fisioterapeuta visualizar todo contexto do paciente, incluindo: histórico de cirurgias com tempo decorrido, objetivos com countdown, patologias ativas/tratadas, evolução de testes com gráficos e tabelas, alertas para medições obrigatórias, e replicação de condutas anteriores.

Estrutura de Dados e Tipos

1. Expandir Tipos Existentes

Arquivo: types.ts

✓ Verificar se Surgery possui todos campos necessários (data, descrição, tempo de recuperação)

✓ Verificar se PatientGoal possui campos para countdown (targetDate, currentProgress)

✓ Verificar se Pathology possui status 'active' | 'resolved'

✓ Verificar se AssessmentTestConfig existe e possui frequência/obrigatoriedade

Criar tipos novos:

SessionEvolution - dados da evolução da sessão

TestEvolutionData - dados para gráficos de evolução

ConductTemplate - template de conduta para replicação

MandatoryTestAlert - alerta de teste obrigatório

2. Services - CRUD Completo

Surgery Service (services/surgeryService.ts)

getSurgeriesByPatientId(patientId: string): Promise<Surgery[]>

addSurgery(patientId: string, surgery: Omit<Surgery, 'id'>): Promise<Surgery>

updateSurgery(surgeryId: string, data: Partial<Surgery>): Promise<Surgery>

deleteSurgery(surgeryId: string): Promise<void>

calculateTimeSinceSurgery(surgeryDate: string): string - calcular tempo em formato legível

Patient Goals Service (services/patientGoalsService.ts)

getGoalsByPatientId(patientId: string): Promise<PatientGoal[]>

addGoal(patientId: string, goal: Omit<PatientGoal, 'id'>): Promise<PatientGoal>

updateGoalProgress(goalId: string, progress: number, currentValue?: string): Promise<PatientGoal>

updateGoal(goalId: string, data: Partial<PatientGoal>): Promise<PatientGoal>

deleteGoal(goalId: string): Promise<void>

calculateCountdown(targetDate: string): { days: number; formatted: string }

markGoalCompleted(goalId: string): Promise<PatientGoal>

Pathology Service (services/pathologyService.ts)

getPathologiesByPatientId(patientId: string): Promise<Pathology[]>

getActivePathologies(patientId: string): Promise<Pathology[]>

getResolvedPathologies(patientId: string): Promise<Pathology[]>

addPathology(patientId: string, pathology: Omit<Pathology, 'id'>): Promise<Pathology>

updatePathology(pathologyId: string, data: Partial<Pathology>): Promise<Pathology>

markAsResolved(pathologyId: string): Promise<Pathology>

markAsActive(pathologyId: string): Promise<Pathology>

deletePathology(pathologyId: string): Promise<void>

Test Evolution Service (services/testEvolutionService.ts)

getTestEvolutionData(patientId: string, testName: string): Promise<TestEvolutionData[]>

getTestHistory(patientId: string): Promise<Map<string, TestEvolutionData[]>>

addTestResult(patientId: string, sessionId: string, result: TestResult): Promise<void>

getMandatoryTests(patientId: string, sessionNumber: number): Promise<AssessmentTestConfig[]>

checkMandatoryTestsCompleted(patientId: string, sessionId: string):
Promise<boolean>

getTestStatistics(patientId: string, testName: string): Promise<TestStatistics>

Conduct Replication Service (services/conductReplicationService.ts)

getSavedConducts(patientId: string): Promise<ConductTemplate[]>

saveConductAsTemplate(patientId: string, conduct: any, name: string):
Promise<ConductTemplate>

replicateConduct(conductId: string): Promise<any>

deleteConduct(conductId: string): Promise<void>

Session Evolution Service (services/sessionEvolutionService.ts)

getSessionEvolution(sessionId: string): Promise<SessionEvolution>

saveSessionEvolution(data: Omit<SessionEvolution, 'id'>):
Promise<SessionEvolution>

updateSessionEvolution(sessionId: string, data: Partial<SessionEvolution>):
Promise<SessionEvolution>

deleteSessionEvolution(sessionId: string): Promise<void>

getEvolutionsByPatientId(patientId: string): Promise<SessionEvolution[]>

Interface de Usuário - 3 Opções de Implementação

Opção 1: Página Nova (pages/SessionEvolutionPage.tsx)

Rota: /atendimento/:appointmentId

Layout fullscreen com navegação por abas

Botão "Voltar para Agenda" no topo

Opção 2: Modal Fullscreen (components/session/SessionEvolutionModal.tsx)

Abre sobre a agenda

z-index alto, cobre toda tela

Botão X para fechar

Opção 3: Expansão da SessionFormPage

Expandir SessionFormPage.tsx existente

Adicionar novas seções e funcionalidades

Manter estrutura de 3 colunas

Implementação: Criar todas 3 opções com toggle no código para escolher depois.

Layout da Interface Principal

Estrutura de 4 Colunas

Col 1 (30%)	Col 2 (25%)	Col 3 (25%)	Col 4 (20%)
Formulário	Histórico	Testes &	Resumo
SOAP	& Condutas	Evolução	Paciente

Coluna 1: Formulário SOAP (30%)

Campo Subjetivo (queixas do paciente)

Campo Objetivo (avaliação do profissional)

Campo Avaliação

Campo Plano/Conduta

Botões: "Replicar Conduta Anterior", "Salvar", "Cancelar"

Coluna 2: Histórico & Condutas (25%)

Componente: SessionHistoryPanel.tsx

Últimas Sessões (cards colapsáveis)

Data, resumo, dor relatada

Botão "Replicar Esta Conduta"

Cirurgias

Lista de cirurgias com data

Badge mostrando "há X dias/meses/anos"

Modal para adicionar/editar cirurgias

Tempo de Tratamento

"Paciente em tratamento há X dias/meses"

Data primeira consulta

Coluna 3: Testes & Evolução (25%)

Componente: TestEvolutionPanel.tsx

Alertas de Testes Obrigatórios

Banner vermelho se há testes pendentes

Lista de testes que devem ser realizados

Ex: "  Medição de amplitude do joelho obrigatória nesta sessão"

Patologias

Seção "Em Tratamento" (status: active)

Seção "Tratadas/Resolvidas" (status: resolved)

Botão para adicionar/editar patologias

Gráficos de Evolução

Seletor de métrica (dropdown)

Seletor de tipo de gráfico (barras, linha, área)

Gráfico interativo com Recharts

Exemplos de métricas:

Amplitude de movimento

Nível de dor (EVA 0-10)

Força muscular

Equilíbrio

Testes funcionais

Tabela de Evolução

Data | Sessão | Métrica | Valor | Variação

Ordenável por coluna

Export para CSV/Excel

Coluna 4: Resumo Paciente (20%)

Componente: PatientSummaryPanel.tsx

Info Básica

Nome, idade, foto

Contato rápido (WhatsApp, telefone)

Objetivos com Countdown

Card para cada objetivo

Barra de progresso

Countdown: "Faltam X dias para [objetivo]"

Exemplos:

"Prova TAF - Correr 1km em 2min | 45 dias restantes"

"Maratona em 20/03/2025 | 120 dias restantes"

Badge de prioridade (baixa/média/alta/crítica)

Botão para adicionar/editar objetivos

Métricas Rápidas

Número de sessões realizadas

Taxa de presença

Próxima sessão agendada

Componentes Principais

1. SessionEvolutionContainer

Arquivo: components/session/SessionEvolutionContainer.tsx

Container principal que gerencia estado

Integra todas as colunas

Gerencia navegação/abas

2. SOAPFormPanel

Arquivo: components/session/SOAPFormPanel.tsx

Formulário SOAP completo

Validação de campos

Auto-save (opcional)

Integração com alertas de testes obrigatórios

3. SessionHistoryPanel

Arquivo: components/session/SessionHistoryPanel.tsx

Lista últimas sessões (10 mais recentes)

Card expandível para cada sessão

Botão "Replicar Conduta"

Modal de detalhes da sessão

4. SurgeryTimeline

Arquivo: components/session/SurgeryTimeline.tsx

Timeline visual de cirurgias

Badge com tempo decorrido

Modal CRUD de cirurgias

5. SurgeryFormModal

Arquivo: components/session/SurgeryFormModal.tsx

Formulário completo: nome, data, descrição, cirurgião, hospital, complicações

Validação de campos

Save/Update/Delete

6. TestEvolutionPanel

Arquivo: components/session/TestEvolutionPanel.tsx

Container para gráficos e tabelas

Seletor de métricas

Alertas de testes obrigatórios

7. MandatoryTestAlert

Arquivo: components/session/MandatoryTestAlert.tsx

Banner vermelho destacado

Lista de testes pendentes

Bloqueio de salvamento se não realizado

Botão "Registrar Exceção" (com log de auditoria)

8. EvolutionChart

Arquivo: components/session/EvolutionChart.tsx

Gráfico interativo com Recharts

Suporta múltiplos tipos: barras, linha, área

Tooltip com detalhes

Zoom e pan

Legenda configurável

9. EvolutionTable

Arquivo: components/session/EvolutionTable.tsx

Tabela responsiva

Ordenação por coluna

Filtros

Export para CSV

Paginação

10. PathologyManager

Arquivo: components/session/PathologyManager.tsx

Lista patologias ativas e resolvidas

Badge de status

Modal CRUD de patologias

11. PathologyFormModal

Arquivo: components/session/PathologyFormModal.tsx

Formulário: nome, CID, data diagnóstico, severidade, região afetada, status

Autocomplete de CID

Save/Update/Delete

12. PatientGoalsPanel

Arquivo: components/session/PatientGoalsPanel.tsx

Lista de objetivos do paciente

Card para cada objetivo com:

Título e descrição

Barra de progresso

Countdown visual

Badge de prioridade

Modal CRUD de objetivos

13. GoalFormModal

Arquivo: components/session/GoalFormModal.tsx

Formulário: título, descrição, categoria, data alvo, valor alvo, valor atual, prioridade

Calculadora de progresso

Validação de datas

Save/Update/Delete

14. GoalCountdown

Arquivo: components/session/GoalCountdown.tsx

Componente visual de countdown

Formatos: dias, semanas, meses

Animação de progresso

Alerta quando próximo da data

15. ConductReplicationDialog

Arquivo: components/session/ConductReplicationDialog.tsx

Lista sessões anteriores

Preview da conduta

Seleção de campos para replicar

Confirmação antes de aplicar

16. TreatmentDurationCard

Arquivo: components/session/TreatmentDurationCard.tsx

Calcula tempo desde primeira sessão

Exibe de forma visual

Link para histórico completo

Alertas e Validações

Sistema de Alertas Obrigatórios

Arquivo: services/mandatoryTestAlertService.ts

Verificar testes obrigatórios antes de salvar

Configuração por patologia (ex: LCA requer medição de amplitude)

Níveis de alerta:

Crítico: Bloqueia salvamento (ex: pós-op LCA sem medição)

Importante: Aviso destacado mas permite salvar

Leve: Apenas notificação

Validações

Data de cirurgia não pode ser futura

Data alvo de objetivo não pode ser passada

Testes obrigatórios críticos bloqueiam salvamento

Valores de testes devem estar em ranges válidos

Relatórios e Sugestões para Médicos

Geração Automática de Insights

Service: services/medicalReportSuggestionsService.ts

Analisar evolução e gerar insights:

"Paciente reduziu dor de 9/10 para 0/10 em 5 sessões"

"Amplitude de movimento aumentou 40% desde avaliação inicial"

"Retornou ao esporte de forma gradual na semana 8"

"Teste de força muscular: evolução de grau 3 para grau 5"

Comparações temporais automáticas

Identificação de marcos importantes

Sugestão de texto para laudo médico

Componente de Sugestões

Arquivo: components/session/MedicalReportSuggestions.tsx

Painel lateral ou seção colapsável

Lista de insights gerados

Botão "Copiar para Relatório"

Preview de como ficará no relatório

Gráficos de Dados Interessantes

Métricas Sugeridas para Gráficos

Evolução da Dor (EVA)

Gráfico de linha

Eixo Y: 0-10, Eixo X: Sessões/Datas

Linha de meta (ex: dor < 3)

Amplitude de Movimento

Gráfico de barras ou linha

Comparação bilateral (esquerda vs direita)

Valores normais como referência

Força Muscular

Gráfico de radar ou barras

Múltiplos grupos musculares

Graus de força (0-5)

Testes Funcionais

Gráfico de linha temporal

Múltiplas métricas no mesmo gráfico

Ex: distância caminhada, tempo de marcha

Frequência de Presença

Gráfico de barras

Presença vs faltas ao longo do tempo

Taxa de adesão

Retorno Gradual ao Esporte

Timeline visual

Marcos: início atividade leve → moderada → intensa → retorno completo

Datas e observações

Configuração de Gráficos

Permitir escolher tipo de gráfico por métrica

Salvar preferências do usuário

Export de gráficos como imagem (PNG/SVG)

Incluir gráficos em relatórios PDF

Fluxo de Uso

Profissional clica em "Iniciar Atendimento" na agenda

Abre interface de evolução (uma das 3 opções)

Sistema carrega automaticamente:

Dados do paciente

Histórico de sessões

Cirurgias e tempo decorrido

Objetivos com countdown

Patologias ativas/tratadas

Testes obrigatórios para esta sessão

Gráficos de evolução

Alerta vermelho aparece se há testes obrigatórios

Profissional preenche SOAP:

Pode replicar conduta anterior

Pode consultar gráficos e histórico

Vê objetivos do paciente em tempo real

Ao salvar:

Sistema valida testes obrigatórios

Se críticos não realizados: BLOQUEIA com modal

Se importantes não realizados: AVISA mas permite

Após salvar:

Atualiza status do agendamento para "Realizado"

Gera insights automáticos para relatório médico

Atualiza gráficos e estatísticas

Registra no histórico

Integração com Sistema Existente

Pontos de Integração

AgendaPage.tsx (linha 337): Botão "Iniciar Atendimento" chama nova interface

AppointmentDetailModal.tsx: Adicionar botão "Iniciar Atendimento"

SessionFormPage.tsx: Pode ser expandida ou substituída

PatientDetailPage: Link para ver evolução completa

Migração de Dados

Verificar se campos já existem no banco

Criar migrations se necessário (Supabase)

Popular dados de exemplo para testes

Testes

Casos de Teste

CRUD Completo:

Criar, ler, atualizar, deletar cirurgias

Criar, ler, atualizar, deletar objetivos

Criar, ler, atualizar, deletar patologias

Criar, ler evolução de sessão

Alertas:

Teste com patologia que requer medição obrigatória

Tentativa de salvar sem realizar teste

Registrar exceção com justificativa

Gráficos:

Carregar dados de evolução

Alternar tipos de gráfico

Filtrar por período

Countdown:

Objetivo com data futura

Objetivo próximo do vencimento

Objetivo vencido

Replicação:

Replicar conduta de sessão anterior

Replicar conduta específica

Arquivos Principais a Criar/Modificar

Novos Serviços (Services)

services/surgeryService.ts

services/patientGoalsService.ts

services/pathologyService.ts

services/testEvolutionService.ts

services/conductReplicationService.ts

services/sessionEvolutionService.ts

services/mandatoryTestAlertService.ts

services/medicalReportSuggestionsService.ts

Novos Componentes (16 componentes)

components/session/SessionEvolutionContainer.tsx

components/session/SOAPFormPanel.tsx

components/session/SessionHistoryPanel.tsx

components/session/SurgeryTimeline.tsx

components/session/SurgeryFormModal.tsx

components/session/TestEvolutionPanel.tsx

components/session/MandatoryTestAlert.tsx

components/session/EvolutionChart.tsx

components/session/EvolutionTable.tsx

components/session/PathologyManager.tsx

components/session/PathologyFormModal.tsx

components/session/PatientGoalsPanel.tsx

components/session/GoalFormModal.tsx

components/session/GoalCountdown.tsx

components/session/ConductReplicationDialog.tsx

components/session/TreatmentDurationCard.tsx

components/session/MedicalReportSuggestions.tsx

Páginas (3 opções)

pages/SessionEvolutionPage.tsx (Opção 1)

components/session/SessionEvolutionModal.tsx (Opção 2)

Expandir pages/SessionFormPage.tsx (Opção 3)

Modificações em Arquivos Existentes

types.ts - adicionar novos tipos

pages/AgendaPage.tsx - adicionar botão "Iniciar Atendimento"

components/AppointmentDetailModal.tsx - adicionar ação

AppRoutes.tsx - adicionar rota (se opção 1)

Ordem de Implementação

Fase 1: Fundação (Services & Tipos)

Expandir types.ts com novos tipos

Criar todos os services com CRUD completo

Testar services isoladamente

Fase 2: Interface Base

Criar SessionEvolutionContainer (as 3 versões)

Criar estrutura de layout de 4 colunas

Integrar com dados mockados

Fase 3: Coluna 1 - Formulário

Criar/adaptar SOAPFormPanel

Integrar com replicação de condutas

Adicionar validações

Fase 4: Coluna 2 - Histórico

Criar SessionHistoryPanel

Criar SurgeryTimeline + CRUD

Criar TreatmentDurationCard

Fase 5: Coluna 3 - Evolução

Criar TestEvolutionPanel

Criar MandatoryTestAlert com lógica de bloqueio

Criar PathologyManager + CRUD

Criar EvolutionChart (múltiplos tipos)

Criar EvolutionTable com export

Fase 6: Coluna 4 - Resumo & Objetivos

Criar PatientGoalsPanel

Criar GoalFormModal com CRUD

Criar GoalCountdown visual

Adicionar métricas rápidas

Fase 7: Funcionalidades Avançadas

Implementar sistema de alertas completo

Criar geração automática de insights

Adicionar MedicalReportSuggestions

Implementar export de gráficos

Fase 8: Integração & Polimento

Integrar botão "Iniciar Atendimento" na agenda

Conectar com dados reais (Supabase)

Testar fluxo completo

Ajustes de UX e responsividade

Documentação

Observações Finais

Manter design consistente com resto do sistema

Usar Tailwind CSS para estilização

Componentes devem ser reutilizáveis

Código bem documentado

Performance: lazy loading de gráficos pesados

Acessibilidade: ARIA labels, keyboard navigation

Mobile: interface responsiva (pode simplificar em mobile)

To-dos

[] Expandir types.ts e criar todos os 8 services com CRUD completo (surgery, goals, pathology, testEvolution, conductReplication, sessionEvolution, mandatoryTestAlert, medicalReportSuggestions)

[] Criar SessionEvolutionContainer nas 3 versões (página, modal, expansão) com layout de 4 colunas responsivo

[] Criar/adaptar SOAPFormPanel com validações e integração com replicação de condutas

[] Implementar coluna 2: SessionHistoryPanel, SurgeryTimeline com CRUD completo, TreatmentDurationCard

[] Implementar coluna 3: TestEvolutionPanel, MandatoryTestAlert com bloqueio, PathologyManager com CRUD, EvolutionChart (barras/linha/área), EvolutionTable com export

[] Implementar coluna 4: PatientGoalsPanel, GoalFormModal com CRUD, GoalCountdown visual, métricas rápidas

[] Implementar sistema completo de alertas obrigatórios e geração automática de insights para relatórios médicos (MedicalReportSuggestions)

[] Integrar botão 'Iniciar Atendimento' na AgendaPage e AppointmentDetailModal, conectar com dados reais, testar fluxo completo, ajustes de UX